Taala App Lock: Master Build Prompt (Android 14+)

A complete, copy-paste prompt for an AI coding agent to build a polished Android app lock app.

PROMPT START

0. Your role and working rules

You are a senior Android engineer and product designer. Build a complete, production-quality Android app lock app. Working name: **Taala**. Applications chosen by the user must open only after authentication (face, fingerprint, or screen lock), and the app must feel premium: instant, smooth, calm, and trustworthy.

Rules for how you work:

- Work milestone by milestone (listed in section 16). After each milestone, make sure the project compiles and runs, then give me a short summary and the exact commands to build and test. Do not move on with a broken build.
- No placeholders, no TODO comments, no fake or mocked logic in the final code. Every screen listed below must really work.
- Write original code. Study the reference projects in section 17 for ideas and architecture, but do not copy their code. Respect their licenses.
- Use the latest stable versions of Kotlin, Android Gradle Plugin, Compose BOM, and AndroidX libraries (check current versions, do not guess old ones).
- If something is impossible on Android without root or system privileges, say so plainly and build the best honest alternative. Never claim protection the app cannot provide.
- Ask me a question only if you are truly blocked. Otherwise choose the sensible default and note the assumption.

1. Product summary

Taala lets the user pick apps (banking, chat, gallery, social, settings, etc.) and locks them. When a locked app is about to open, Taala instantly covers it with an opaque, screenshot-proof screen and asks for authentication through the system BiometricPrompt. After a successful unlock, the app stays unlocked while the user is using it, and locks again when they leave, after a delay they choose.

Target users: ordinary phone users in India and worldwide who want a fast, private, ad-free app lock. No account, no cloud, no internet, no tracking, no ads.

Platform: **minSdk 34 (Android 14)**, compileSdk and targetSdk = the latest stable API level. Must work on Android 14, 15, 16 and newer, on phones, foldables and tablets.

2. Honest technical reality (build around this)

Android has no public API to lock another app. Every app locker works by (a) detecting which app is coming to the foreground and (b) covering it before the user sees content. Taala must implement **two engines** and switch between them automatically:

Engine A: Accessibility engine (default, preferred). An AccessibilityService listens to window events. When the user taps a locked app's icon, Recents card or search result, or when a locked app's window appears any other way (split screen, picture-in-picture, link, notification), Taala immediately shows an opaque cover using a TYPE_ACCESSIBILITY_OVERLAY window, then launches the lock screen activity. This covers the app from the tap, before the app has drawn anything.

- [ ] **Engine B: Compatibility mode (fallback).** Used when accessibility is unavailable (for example the user refuses it, or Android's Advanced Protection mode reserves accessibility for genuine accessibility tools). A foreground service polls UsageStatsManager usage events (about every 150 ms while the screen is on) to see which app resumed, and draws a cover with SYSTEM_ALERT_WINDOW. It is slightly slower (a warm launch can show the app for about half a second) and cannot offer self-protection. It needs a low-priority foreground notification.

The app switches between engines on its own, including the moment Advanced Protection is turned on or off (use the QUERY_ADVANCED_PROTECTION_MODE permission and the related callback/API where available). Show the user which engine is active in Settings and Home.

Taala must never declare itself an accessibility tool: set `isAccessibilityTool="false"`. It must be written to comply with Google Play policy so Play Protect has nothing to flag.

Note: Android 17 QPR2 adds a native App Lock, but turning it on needs the LOCK_APPS permission that only the default launcher can hold, so a normal app cannot use it. Do not rely on it. Detect and mention it in the About screen only if it is easy.

3. Tech stack (fixed, do not substitute)

- Language: Kotlin only. Coroutines and Flow.
- UI: Jetpack Compose with Material 3 (use Material 3 Expressive components and motion where the stable library offers them). No XML screens, except the minimum XML needed for the accessibility service config, backup rules, and the launcher icon.

- Architecture: single-module is fine, but with clean package separation (section 4). MVVM with unidirectional data flow: ViewModel, StateFlow, immutable UI state classes.
- Dependency injection: Hilt (or Koin if Hilt creates trouble with services). Pick one and be consistent.
- Storage: Room for locked apps, profiles, schedules and the intruder log. DataStore (Preferences or Proto) for settings. Sensitive values (optional app PIN hash, backup keys) protected with Android Keystore.
- Navigation: Navigation Compose with predictive back support.
- Images: app icons loaded from PackageManager into a small in-memory cache (no network image library needed).
- Build: Gradle Kotlin DSL, version catalog (libs.versions.toml), R8 minify and resource shrinking in release, baseline profile and startup profile.
- Permissions: **no INTERNET permission. no QUERY_ALL_PACKAGES.** No analytics, no ads, no third-party SDKs.

4. Project structure

```
app/src/main/java/<package>/
  core/data        Room database, DAOs, repositories, DataStore settings
  core/security    SecurityVault (settings, locked apps, profiles),
LockStateTracker (sessions),
                   AuthManager (BiometricPrompt), ProtectionEngine (which
engine runs),
                   AdvancedProtection, WindowProtection, BackupManager,
KeystoreHelper
  core/diagnostics DiagnosticLog (on-device 3-day log, no app names in
release logs)
  core/receiver    BootReceiver, ScreenReceiver
  feature/lock     TaalaAccessibilityService, UsageWatchService (compat
mode), ForegroundRules,
                   SessionRules, LockLauncher, CoverOverlay,
LockScreenActivity,
                   RecentsWatcher, SelfProtectionDetector, AppLabelCache
  feature/home     Home screen, app list, search, filters
  feature/profiles Profiles and profile editor
  feature/schedule Time-based lock schedules
  feature/intruder Intruder log and selfie capture
  feature/settings Settings, pause, backup/restore, about, diagnostic log
  feature/setup    Onboarding and permission wizard with prominent
disclosures
```

```
feature/notification  Notification shield (NotificationListenerService)
feature/tile     Quick Settings tile
ui/theme         Colors, typography, shapes, motion tokens, haptics
ui/components    Reusable composables
```

5. Locking logic (the core, get this exactly right)

5.1 Covering. Cover must be opaque, use FLAG_SECURE (no screenshots or screen recording), and show the locked app's icon and name on a branded background. It goes up immediately, then LockScreenActivity takes over and shows the system BiometricPrompt.

5.2 Authentication. Use androidx.biometric BiometricPrompt with `BIOMETRIC_STRONG or DEVICE_CREDENTIAL`. Class 3 face unlock and fingerprint work at the same time; weak (Class 2) face unlock is never accepted. If the user has no biometrics or screen lock enrolled, guide them to Settings during setup and refuse to enable locking until one exists. Optional extra: an app-specific PIN (4 to 8 digits) stored as a salted PBKDF2 hash wrapped with a Keystore key, with escalating lockout delays after failed attempts (30 s, 1 min, 5 min, 15 min).

5.3 Sessions. An unlocked app stays unlocked for as long as any of its windows is on screen, whatever happens inside it (refreshes, dialogs, keyboards, chat heads, sign-in sheets, assistants). It counts as left only when its windows are gone. Then it re-locks after the re-lock delay, chosen per profile: immediately, 30 seconds, 1 minute, 5 minutes, 15 minutes, or when the screen turns off.

5.4 Helper-app hops. Going directly to a helper app and back within 5 minutes must not end the session, even across several hops in a row. Helpers: authenticator apps, email, photo or file pickers and galleries, camera, Settings, Play Store purchase/review/update pages, permission request dialogs, share sheets, and interruptions like calls and alarms. Detect picker/camera/file apps by asking PackageManager which apps handle the relevant intents (ACTION_PICK, ACTION_GET_CONTENT, ACTION_IMAGE_CAPTURE, OpenDocument) rather than hard-coding package names.

5.5 Lock everything. Screen off, screensaver start, device lock, and Lockdown from the power menu always end all sessions.

5.6 Things that must never be covered. The home screen in use (launcher) and the system launcher's Recents must never be covered, so the user can always press Home. Those apps cannot have their lock switch turned on. Never lock the phone dialer's emergency call screen or the system UI. Never lock Taala's own accessibility settings page unless self-protection is on.

5.7 Special windows. Handle split screen, picture-in-picture (cover with a patch over the PiP window), freeform/desktop windows, external displays, and foldable cover vs main screens. A locked app's lock screen should take the whole display or send the user Home

so the app is never usable beside the prompt.

5.8 Work profile and other profiles. Compatibility mode cannot see apps in other profiles (Android limitation). Accessibility engine still locks them. Document this in the app.

6. Accessibility service details

- Declare `isAccessibilityTool="false"`, `canRetrieveWindowContent="false"` (switch it on only if a feature truly needs window lists or Recents card names, and then update the disclosure text), `accessibilityEventTypes` limited to window state and windows changed, a short `notificationTimeout`, and `flagReportViewIds` off.
- The service never reads text the user types and never reads screen contents beyond what a listed feature needs. Say exactly that in the disclosure.
- Keep the service lightweight: no heavy work on the event callback, no allocations in hot paths, use a single-thread dispatcher for engine decisions.
- Inside Taala's own screens, mark views accessibility-data-sensitive (Android 14 View API `setAccessibilityDataSensitive`) so only genuine accessibility tools like TalkBack can read or operate them. This stops accessibility malware from tapping "Pause" or reading locked-app info.

7. Features (all required)

1. **App list and locking.** Show all launchable apps with icon and name, search, sort (A to Z, recently installed, locked first), filter (All, Locked, Unlocked, System, User). One-tap toggle per app. Smart suggestions: banking, wallets, messengers, gallery, Settings, Play Store, browsers, file managers shown in a "Suggested to lock" row.
2. **Profiles.** Built-in Finance, Social, Work, and custom profiles. Each profile has its own re-lock delay and on/off switch. An app is locked when its own switch is on and its profile is on. An app lives in at most one profile. Optional auto-sort of recognized apps into built-in profiles.
3. **Schedules (focus mode).** Time-based auto-locking: pick days and time ranges (for example school hours or sleep time) when a profile is forced on or off. Uses AlarmManager exact alarms only if permission is granted, otherwise WorkManager inexact timing, and explains the difference.
4. **Notification shield.** Optional. A locked app's notifications are snoozed and replaced by one stand-in "N hidden notifications"; tapping the stand-in asks for unlock then opens the original. Calls, alarms, media and navigation notifications always show. Requires notification access with a clear disclosure.
5. **Intruder log.** After a configurable number of failed attempts (default 3), record time, app name and an optional front-camera photo, stored encrypted only on the device. Needs CAMERA permission, asked only when the user turns this on. Show a timeline in

the Intruder screen with delete and "delete all".

6. **Taala locks itself.** Opening Taala needs authentication, and it locks again when the user leaves it or after 3 minutes of inactivity.
7. **Self-protection (optional, off by default, accessibility engine only).** Taala's App info page (force stop, uninstall, clear data), its accessibility switch, notification access and the uninstall dialog require authentication. Detection is heuristic (package + content), so test on Pixel, Samsung and Xiaomi layouts and say it may not work on every manufacturer's Settings.
8. **Protection alert.** If protection stops (service switched off, compat mode loses a permission), post a notification with a one-tap fix.
9. **Pause.** Pause for 15 minutes, 1 hour, or until turned back on, from the app and from a Quick Settings tile (the pause itself needs authentication).
10. **Backup and restore.** Export locked apps, profiles, schedules and settings to a file the user picks (Storage Access Framework), encrypted with AES-256-GCM using a key derived from a user password with PBKDF2-SHA256 (high iteration count, random salt). Restore after reinstall or on a new phone. Android cloud backup and device transfer stay disabled.
11. **Diagnostic log.** On by default, can be turned off, keeps Taala's own lock decisions for 3 days on the device, user can share it from Settings. In release builds do not log app names or package names of locked apps.
12. **Disguise (optional extra).** Alternative app icon/name (for example Calculator-style) using activity-alias switching, off by default.
13. **Languages.** English and Hindi, fully translated, with all strings in string resources. Support RTL and large font scaling.

8. Permissions and consent

Every sensitive permission needs a **prominent in-app disclosure screen first**. Only an explicit **Agree** button continues; Back or dismissing is not consent. Explain in plain language what the permission does, what is not collected, and how to turn it off.

Permission	Why
Accessibility service	See which app is on screen and cover locked apps. Does not read what you type.
USE_BIOMETRIC	Face and fingerprint through the system prompt

Permission	Why
HIDE_OVERLAY_WINDOWS	Hide other apps' overlays while Taala's screens show (tapjacking defence)
POST_NOTIFICATIONS	Protection alerts and notification-shield stand-ins
RECEIVE_BOOT_COMPLETED	Restart compat mode or warn after reboot
QUERY_ADVANCED_PROTECTION_MODE	Detect Advanced Protection and switch engines
PACKAGE_USAGE_STATS, SYSTEM_ALERT_WINDOW, FOREGROUND_SERVICE, FOREGROUND_SERVICE_SPECIAL_USE	Compatibility mode only
Notification access (optional)	Notification shield
CAMERA (optional)	Intruder selfie
SCHEDULE_EXACT_ALARM (optional)	Exact schedules

On Android 14+, a foreground service of type specialUse must declare its subtype property in the manifest. Do it correctly. Package visibility must use a narrow `<queries>` block (launchable apps, launchers, keyboards, picker/camera apps, Settings), not QUERY_ALL_PACKAGES.

Sideloaded builds on Android 13+ may be blocked from enabling accessibility until "Allow restricted settings" is turned on in the app's App info page. The setup wizard must detect this and show a clear illustrated step-by-step for it, including the exact path (App info, three-dot menu, Allow restricted settings).

9. Security hardening checklist

- FLAG_SECURE on every Taala window and on the cover.
- Reject touches through obscuring windows (`filterTouchesWhenObscured`) and use `Window.setHideOverlayWindows(true)` while Taala screens show.
- `android:allowBackup="false"` and data-extraction rules that exclude everything.
- All components `exported="false"` unless they must be reachable (launcher activity, QS tile with the required permission).
- No logging of locked app names in release (strip Log.v/d/i/w with R8, and never put app names in Log.e).

- No INTERNET permission, no cleartext traffic, no WebView.
- Taala's screens do not appear in Recents thumbnails.
- Lock screen survives rotation, fold/unfold, split-screen, dark/light switch, process death, and low-memory kill without ever exposing the locked app.

10. UI and UX specification (this is what makes it feel premium)

Design language. Material 3 Expressive, calm and modern. Dynamic color (Material You) on Android 12+ with a branded fallback palette: deep indigo primary, soft teal accent, warm neutral surfaces. Light, dark and true-black AMOLED themes, with a Follow-system default. Large rounded shapes (28 dp cards, 20 dp chips), generous spacing (multiples of 4 dp), a clear typographic hierarchy (Display for hero numbers, Title for sections, Body for content). Edge-to-edge always (Android 15 enforces it), correct insets, and predictive back animations.

Motion. Everything uses spring or emphasized easing, never abrupt jumps. Toggle switches animate and give a light haptic tick. Cards press in with a subtle scale. List items enter with a short staggered fade-and-slide. Shared-element transition from app row to its detail sheet. Lock screen entrance: cover fades in under 100 ms, then the app icon scales in with a spring. Success unlock: small check animation and a soft haptic, then dissolve to the app. Failure: gentle horizontal shake plus error haptic. Respect the system "remove animations" setting.

Screens (build all):

1. **Splash.** Android 12 SplashScreen API with the animated key icon, no fake loading delay.
2. **Onboarding (3 pages).** (a) What Taala does, (b) how it works and what it cannot do (honest limits), (c) privacy promise: no internet, no data collected. Skip option. Pager with animated dots.
3. **Setup wizard.** Step list with live status chips: Biometric/screen lock check, Accessibility (with disclosure, restricted-settings help, and a button that opens the exact settings page), Notifications, optional extras. Each step shows Done, Needed or Optional. The wizard rechecks status when the user returns from Settings.
4. **Home.** Large top bar that collapses on scroll. Status card at top: protection state (Active, Paused, Needs attention), engine in use, number of locked apps. Below: suggested apps row, search field, filter chips, then the app list with toggles. Floating "Pause" action. Empty and loading states with illustrations (draw simple vector illustrations).
5. **Profiles.** Cards with colored icon, app count, delay and master switch. Tap to open the profile editor page: name, color, icon, re-lock delay picker, app picker with multi-select and bulk actions.

6. **App detail bottom sheet.** App icon and name, lock toggle, profile selector, per-app re-lock override, notification shield toggle for that app, last unlocked time.
7. **Lock screen.** Full-screen, opaque, brand-colored gradient derived from the locked app's icon color when possible, large app icon, app name, a "Unlock" button that re-triggers BiometricPrompt, a "Use PIN" fallback, and a small "Taala" mark at the bottom. The biometric prompt appears automatically. Back press sends the user Home, never to the locked app.
8. **Schedules.** List of schedules with day chips and time ranges, add/edit sheet with a time picker, enable switch.
9. **Intruder log.** Timeline cards with photo thumbnail (blurred until tapped), time, app. Empty state explains the feature.
10. **Settings.** Grouped sections: Protection (engine, self-protection, pause defaults), Lock behavior (default re-lock delay, lock on screen off), Notifications, Appearance (theme, dynamic color, AMOLED, language), Security (app PIN, disguise icon), Backup and restore, Diagnostics, Help (restricted settings guide, OEM battery guides), About (version, licenses, privacy policy, open-source notices).
11. **Help and troubleshooting.** Illustrated guides for Xiaomi/Redmi/Poco (Autostart, Battery saver: No restrictions), Samsung (remove from Sleeping apps), OnePlus/Oppo/Realme/Vivo (Auto-launch, background activity), and Pixel. Shown only for the detected manufacturer, with a button that opens the right Settings page where possible.
12. **Quick Settings tile.** Toggle pause/resume with authentication.
13. **Widgets.** None required.

Accessibility of Taala itself. Full TalkBack labels, minimum 48 dp touch targets, 4.5:1 contrast, works at 200% font scale, no information by color alone.

Tablets and foldables. Adaptive layouts using window size classes (list-detail on wide screens), and the lock screen correct on every screen size.

Copywriting. Short, friendly, plain sentences. Never scary. Explain permissions honestly. Provide English and Hindi.

11. Performance targets

- Cold start to Home under 600 ms on a mid-range phone (use baseline and startup profiles, avoid work in Application.onCreate).
- Accessibility engine: cover drawn within about 100 ms of the tap on a locked app's icon.
- Home list scrolls at a steady 60 fps with 300+ apps (lazy lists, stable keys, cached icons, off-main-thread loading).

- Battery: accessibility engine event-driven with negligible drain. Compat mode only polls while the screen is on and a locked app could appear.
- No wake locks. No memory leaks (verify with LeakCanary in debug builds).

12. Known limits (show honestly in the app's Help screen)

- Root, ADB or a modified OS can bypass any app lock.
- Recents previews of locked apps cannot be blanked by a normal app (only a system app lock can).
- Safe mode disables every downloaded app, so preinstalled locked apps open without a check.
- Some banking apps hide their windows from accessibility; they are covered only when opened by tapping their icon or Recents card.
- An accessibility shortcut assigned to Taala's service can switch it off without authentication, so warn the user while a shortcut is assigned.
- Compat mode cannot see work-profile apps and covers warm launches a little late.
- Closing an app from Recents ends its session only with the accessibility engine.

13. Edge cases that must be handled

- Process death while a lock screen is up: on restart the app must still be covered.
- Rapid switching between Recents and a locked app must never expose its UI (a known bug in other app lockers).
- Two locked apps opened in split screen, PiP over another locked app, an app opened by a notification tap, an app opened by an external link, a locked app launching another locked app.
- Locked app that is uninstalled or updated (clean the database, keep settings).
- New app installed: optional "lock newly installed apps automatically" setting.
- Time change, time zone change, and DST: schedules still correct.
- Biometric enrollment changed or removed: re-check status, never silently unlock.
- Taala's own update or restart: protection resumes after boot and after package replace.
- Device with no biometrics: PIN/credential only.
- Multiple users and Private Space: behave safely, document what is not covered.

14. Testing requirements

- Unit tests (JUnit, Turbine, Robolectric) for session rules, re-lock delay logic, helper-app hops, profile resolution, schedule logic, backup encrypt/decrypt round trip, and the engine switch.
- A "lock-screen gantlet": tests proving that once the lock screen is up it stays up until the user unlocks or leaves, on phone, foldable and tablet sizes, through rotation, split-screen and back/home.
- Compose UI tests for Home, Setup wizard and Settings.
- A manual QA checklist file (QA.md) listing every scenario above with expected result, for a real phone.
- Optional: an adb-driven script that opens locked test apps in different ways and checks the cover appears.
- Lint must pass; lint errors fail the build. Release build must be tested after R8.

15. Release and documentation

- Signing config read from a git-ignored key.properties (with a key.properties.example). Never commit keystores.
- README.md (features, how it works, limits, build steps), CHANGELOG.md, PRIVACY.md (no data collected), SECURITY.md, LICENSE (Apache-2.0 unless I say otherwise), and open-source notices screen.
- A short docs/RELEASING.md covering Google Play declarations (accessibility prominent disclosure and consent, foreground service special use justification, usage access justification), and F-Droid or direct APK distribution.
- Produce a debug APK and a signed release APK/AAB with exact commands.

16. Milestones (build in this order, stop after each for my review)

- **M0:** Project setup, version catalog, theme, navigation shell, CI-ready Gradle, empty screens, splash.
- **M1:** Data layer (Room, DataStore), app list loading, Home screen UI with search, filters and toggles.
- **M2:** Authentication (BiometricPrompt, optional PIN, lockout) and LockScreenActivity with final design.
- **M3:** Accessibility engine, cover overlay, session rules, re-lock delays, helper-app hops. Test with real apps.
- **M4:** Setup wizard and prominent disclosures, restricted-settings help, status checks.
- **M5:** Profiles, app detail sheet, per-app overrides.

- **M6:** Compatibility mode (usage access, overlay, foreground service) and automatic engine switching.
- **M7:** Self-lock of Taala, window hardening, self-protection, protection alert, pause and QS tile.
- **M8:** Notification shield, intruder log, schedules.
- **M9:** Backup/restore, diagnostic log, disguise icon, Hindi translation.
- **M10:** Motion and haptics polish, adaptive layouts, TalkBack pass, OEM help guides.
- **M11:** Tests, gantlet, performance profile (baseline profile), lint, R8 release build.
- **M12:** Docs, QA checklist, signing, final APK/AAB.

Definition of done: every feature above works on a real Android 14 and Android 15+ device, no crashes, no exposed locked app in any listed edge case, smooth 60 fps UI, release build tested, and all docs written.

17. Reference projects (study the ideas, do not copy code)

- **BioKey (github.com/Peacetothemountain/BioKey, Apache-2.0).** Best overall reference: dual engine (accessibility plus compatibility mode), session and helper-app hop rules, profiles, notification shield, self-protection, window hardening, backup, gantlet tests, latency tracing, baseline profile, Play-compliant disclosures.
- **OpenLock (github.com/MalicKAbdullah/openlock).** Reference for the Usage Access plus overlay approach, Device Admin uninstall protection, focus schedules and intruder log, Flutter UI with a native Kotlin lock screen.
- **AppLock by PranavPurwar (github.com/PranavPurwar/AppLock, MIT, archived in October 2026).** Reference for a modern Material 3 look and biometric flow. Its issue tracker shows a Recents-switch bug where the lock overlay briefly drops on Android 15; make sure Taala does not have it.
- **mimbasher/app-lock (github.com/mimbasher/app-lock).** Small clean Kotlin and Compose example of accessibility engine with `canRetrieveWindowContent=false`, usage-access poller fallback and DataStore for locked packages.

PROMPT END